Evidence-Oriented Adversarial Auditing for AI Agents

System design, observable oracles, replay, statistical reporting, and CI integration

Author: Subhodip Dutta
Publisher: RL Hacking Audit
Document type: Technical report / system description
Status: Draft; not peer-reviewed. No benchmark results are claimed.
Version: 1.0 **Date:** 22 October 2026
Repository: https://github.com/Subh10/rl-hacking-audit

Abstract

AI agents combine natural-language interfaces, retrieval, tools, and policy logic, creating failure modes difficult to capture with static checks or subjective model-judge scores. This paper presents RL Hacking Audit (AI Auditor), an open-source framework designed to make adversarial tests more reproducible and reviewable through scenarios, per-attempt canaries, observable-behavior oracles, payload transformations, replay, statistical summaries, structured reports, and CI integration. Its built-in demonstration uses simulated agents and is not evidence that any production model is vulnerable or secure.

1. Introduction

Agent systems consume untrusted content, call tools, query retrieval systems, and reconcile instructions from multiple sources. A useful security test should be rerunnable, inspectable, and repeatable after a mitigation. RL Hacking Audit focuses on repeatable adversarial evaluation and CI workflows; it does not claim a finite test suite proves safety.

2. Design goals

The framework aims to preserve scenario and attempt context; use observable signals where possible; mark unsupported scenarios as skipped; support replay and regression checks; and emit JSON, Markdown, HTML, and SARIF artifacts.

3. Architecture and workflow

A typical workflow selects scenarios, configures an authorized target, executes bounded attempts with a fixed seed, inspects findings and skipped scenarios, saves reports, replays selected attempts after mitigation, and gates changes in CI. Target paths include simulated fixtures, Python callables, HTTP endpoints, and provider-compatible targets. Provider use may incur charges; use least-privilege credentials.

A reproducible run should record the repository revision, Python/runtime version, operating system, target and model version, scenario set, seed, trial count, converter configuration, oracle definitions, skipped tests, and command exit code. Results from different corpora or target configurations should not be compared as if they were directly equivalent.

4. Observable oracles

An oracle determines whether an attempt produced evidence of a failure. Examples include a per-attempt canary appearing in output, a forbidden tool event, or a configured exfiltration indicator. Split canaries can reduce false positives when a target merely quotes an attack. No oracle is universal: canaries miss some semantic failures, token rules can be brittle, and tool-policy checks depend on instrumentation.

5. Payload variation and replay

Converters vary payload form to test whether defenses depend on surface text. Transformed payloads are not guaranteed semantically equivalent. Replay re-executes stored attempts, but outcomes may vary with model versions, sampling, retrieval state, and tool responses. A replay result should therefore include the target and environment context needed to interpret differences.

6. Statistical reporting

Attack-success summaries and confidence intervals help characterize observed runs but do not remove bias, correlation, corpus-selection effects, or distribution shift. A stronger empirical evaluation should specify target versions, scenario selection, seeds, trials, converters, exclusions, oracle definitions, and comparison procedures; report per-scenario results; and avoid comparing incompatible test corpora.

7. CI integration

The GitHub Action makes scans available in workflows and can produce structured artifacts and configured gates. Passing automated tests or an installation smoke test supports basic software/integration confidence, not an independent security audit or proof of detection quality. Record the exact release, commit, and workflow run when publishing claims.

8. Reproducible demonstration

From a clean checkout, install the package in an isolated Python environment and run:

```
python -m pip install .
ai-auditor demo --out demo-reports
```

The built-in demo uses simulated agents, requires no provider API key, and makes no live-model calls. It demonstrates the scanner and report workflow only. Preserve the revision and environment when sharing results; do not call it a real-world exploit or zero-day. Verify the current README and CLI help against the exact release before using these commands in a public demo.

9. Limitations and responsible use

The scenario corpus is a seed set, not an exhaustive threat model. Results depend on adapter behavior, scenario coverage, oracle correctness, and model nondeterminism. A clean scan only means selected tests did not produce findings under tested conditions. Reports may contain sensitive prompts or outputs; protect them, set attempt/spending limits, and test only systems you are authorized to assess.

10. Future evaluation

Future work should measure oracle precision and recall on labeled cases, repeated-run stability, false-positive/negative rates, replay fidelity across model updates, adapter behavior against real providers, and whether CI gates catch regressions without blocking benign changes.

11. Conclusion

RL Hacking Audit offers a workflow for adversarial tests that can be rerun, inspected, compared, and integrated into development pipelines. Its defensible contribution is an evidence-oriented testing workflow, not a claim that one scanner establishes AI safety.

References

1. RL Hacking Audit repository: https://github.com/Subh10/rl-hacking-audit

2. GitHub Actions documentation: https://docs.github.com/actions

3. OASIS, SARIF 2.1.0: https://docs.oasis-open.org/sarif/sarif/v2.1.0/sarif-v2.1.0.html

4. OWASP Top 10 for LLM Applications:

https://owasp.org/www-project-top-10-for-large-language-model-applications/

Author note: This is a technical system-description draft, not peer-reviewed research. No benchmark results or claims of production-model vulnerability are made. Before making empirical claims, run and document a controlled evaluation with labeled cases and exact environment details.